Supplemental Information for "PoolParty: Declartive design of commplex oligonucleotide libraries"

[Author Names]

January 1, 2026

Contents

1 Introduction

PoolParty makes it easy to design complex libraries involving barcodes, mutagenesis, insertion and deletion scanning, and other common operations.

The core idea behind PoolPart is that library construction should be *declarative* rather than *imperative*. Rather than explicitly constructing each sequence in the library, users specify high-level operations that, when carried out in series, generated the desired library. After laying out these rules, users simiply specify how many sequence to generate and poolparty takes care of the rest. This approach separates the *what* (library specification) from the *how* (enumeration or sampling),

Poolparty has two core types of objects: Pools and Operations. A Pool object represents a yet-to-be-realized set of sequences. An Operation object represents a computation that yields a Pool object, and can take zero, one, or multiple Pool objects as input. To create a sequence library, users first specify the rules for making the library by chaining together a series of Operations, the result of which is a Pool boject. Users then call on that pool to generate the number of desired sequences.

The following two examples illustrate PoolParty's core capabilities.

1.1 Example 1: Fixed and Random Mode Operations

Each component Operation of the computation graph can be in one of four modes: `'fixed'`, `'random'`, `'sequential'`, or `'hybrid'`. Fixed mode operations are deterministic: for a given input they always produce the same output. Random mode operation, on the other hand, will generally produce a different output each time they are called due to them having an internal state that changes unpredictably.

Here is an example of PoolParty code that uses `'random'` and `'fixed'` operations to generate the type of construct commonly used in massively parallel reporter assays (MPRAs). First the core sequence elemenets are defined as simple python strings: `wt_seq` is a regulatory sequence of iterest, `adapter_5_seq` is a constant-sequence adapter, and `adapter_3_iupac` is an adapter containing degerate nucleotides that will be used as a barcode. Next, the `pp.mutagenize()` operation is called on `wt_seq`, generating a pool `mut_pool` comprising randomly mutated versions of `wt_seq`. Next, the `pp.from_iupac_motif()` operation is called on `adapter_3_iupac`, generating a pool `adapter_3_pool` in which the `N` characters have been replaced by randomly selected DNA nucleotides. Next, the `pp.join` operation deterministically joins the 3ádaptor sequence, mutagenized pool, and barcode-containing 5ádaptor sequence into a final `oligo_pool_1`. Finally, the `pp.generate_seqs()` method is called with the desired number of sequence. This produces a pandas dataframe `df` that lists each sequence along with a "design card", i.e. a series of fields that describes the decisions each operation made in order to arrive at each resulting sequence.

```
import poolparty as pp

# Mark changes by changing case of affected bases
pp.set_default('mark_changes',True)

# Define component sequences
wt_seq  = 'ATCGATCGATATCGATCGAT'         # 10 nt wild-type
ad5_seq = 'GAAC'                         # 3 nt 5' adapter
ad3_seq = 'TGGANNNNNCCAG'                # 3 + 5N barcode + 3
```

```python
# Create pool of randomly mutagenized wt_seq sequences
mut_pool = pp.mutagenize(wt_seq, num_mutations=3)

# Create pool of 3' adaptors with random barcodes
ad3_pool = pp.from_iupac_motif(ad3_seq, mode='random')

# Join into final oligo pool
oligo_pool = pp.join([ad5_seq, mut_pool, ad3_pool], spacer_str=' ')

# Sample 5 sequences
df = oligo_pool.generate_seqs(num_seqs=5, seed=42)
print(df['seq'])
```

Key points:

- `pp.set_default('mark_changes',True)` sets the default option to mark all changes (mutations) in the generated sequences, typically displaying them in a different case for easy identification.
- `pp.mutagenize()` generates a pool of sequences containing different combinations of mutations at within a specified wild-type sequence. Here we instruct poolparty to introduce 3 mutations into each sequence. This operation runs in random mode by default, which means that the specific mutations that each sequence recieves is random.
- `pp.from_iupac_motif()` generates a pool of sequences derived from an IUPAC sequence, i.e., a DNA sequence in which special characters (such as Ń) signify degenerate bases. This too runs in random mode by default.
- `pp.join()` joins together multiple sequences/Pools. It has no variable internal state so always runs in fixed mode. It also takes an argment, `spacer_str`, which we use here to add a space between the mutagenized region and adaptor sequences.
- `generate_seqs(num_seqs=N, seed)` instructs `oligo_pool` to construct N sequences using the specified random seed. It retuns a Pandas dataframe, here called `df`. The first column is named `'seq'` and contains the resulting sequences:

```
0    GAAC AgCGATCGATATCatTCGAT TGGAatatgCCAG
1    GAAC AgCGATCGATtTCGATCGAc TGGAagaatCCAG
2    GAAC ATCGATCGtTtTCGAcCGAT TGGAcgctaCCAG
3    GAAC ATCGATCGcTgTCGtTCGAT TGGAacaacCCAG
4    GAAC ATCGAgCGATATCGAgCGgT TGGAacaccCCAG
Name: seq, dtype: object
```

- The other columns of the dataframe contain design card information, i.e., the information that was used to create each sequence variant (Note: columns have been pruned and renamed for simplicity).

	seq	iupac_state	positions	wt_chars	mut_chars
0	GAAC AgCGATCGATATCatTCGAT TGGAatatgCCAG	206	(1, 13, 14)	(T, G, A)	(g, a, t)
1	GAAC AgCGATCGATtTCGATCGAc TGGAagaatCCAG	131	(1, 10, 19)	(T, A, T)	(g, t, c)
2	GAAC ATCGATCGtTtTCGAcCGAT TGGAcgctaCCAG	412	(8, 10, 15)	(A, A, T)	(t, t, c)
3	GAAC ATCGATCGcTgTCGtTCGAT TGGAacaacCCAG	65	(8, 10, 14)	(A, A, A)	(c, g, t)
4	GAAC ATCGAgCGATATCGAgCGgT TGGAacaccCCAG	69	(5, 15, 18)	(T, T, A)	(g, g, g)

- During the execution of `oligo_pool.generate_seqs()`, each sequence is computed one-at-a-time using a computation tree that can be visualized by running `oligo_pool.print_tree()`:

```
pool[4] (pool, n=1)
└── op[4]:join [op=join, mode=fixed, n=1]
    ├── pool[3] (pool, n=1)
    │   └── op[3]:from_seq [op=from_seq, mode=fixed, n=1]
    ├── pool[1] (pool, n=1)
    │   └── op[1]:mutagenize [op=mutagenize, mode=random, n=1]
    │       └── pool[0] (pool, n=1)
    │           └── op[0]:from_seq [op=from_seq, mode=fixed, n=1]
    └── pool[2] (pool, n=1)
        └── op[2]:from_iupac_motif [op=from_iupac_motif, mode=random, n=1]
```

1.2 Example 2: Sequential Mode Operations

In many experimental contexts, such as systematic mutagenesis studies or deletion scanning, users need to enumerate *all* possible sequence variants rather than randomly sample from them. This requires `'sequential'` mode operations, which cycle through a finite set of internal states in a deterministic order.

A key challenge arises when multiple sequential operations are combined: how should their internal states advance as sequences are generated? For instance, if one operation scans through 4 positions and another cycles through 2 insert variants, should all $4 \times 2 = 8$ combinations be enumerated (Cartesian product), or should the operations advance in lockstep yielding only 4 sequences (synchronized iteration)?

PoolParty addresses this challenge using an underlying library called StateCounter. Each sequential operation maintains an internal "counter" that tracks its current state. When operations are combined, their counters are composed using algebraic rules that determine how states propagate during sequence generation. The two most common compositions are:

- **Cartesian product**: Enumerate all combinations of states from multiple operations. If operation A has m states and operation B has n states, the product has $m \times n$ states.
- **Stack (union)**: Treat operations as alternatives, activating one at a time. If A has m states and B has n states, the stack has $m + n$ states.

The following example demonstrates these concepts. We perform a replacement scan (inserting either `'AAAAA'` or `'TTTTT'` at each of 4 positions) and a deletion scan (deleting 5 nucleotides at each of the same 4 positions), then stack these two scanning operations together.

```python
# Create pool of two insert variants (2 states)
ins_pool = pp.from_seqs(['AAAAA', 'TTTTT'], mode='sequential')

# Replacement scan: 2 inserts x 4 positions = 8 states (Cartesian product)
rep_pool = pp.replacement_scan(
    wt_seq, ins_pool,
    positions=range(0, 20, 5),  # [0, 5, 10, 15]
    mode='sequential'
)

# Deletion scan: 4 positions = 4 states
del_pool = pp.deletion_scan(
    wt_seq,
    deletion_length=5,
    positions=range(0, 20, 5),
    mode='sequential'
)

# Stack the two scans: 8 + 4 = 12 states (union)
combo_pool = pp.stack([rep_pool, del_pool])

# Join with adapters (reusing ad5_seq and ad3_pool from Example 1)
oligo_pool_2 = pp.join([ad5_seq, combo_pool, ad3_pool], spacer_str=' ')

# Enumerate all 12 states exactly once
df = oligo_pool_2.generate_seqs(num_complete_iterations=1, seed=42)
print(df['seq'])
```

Key points:

- `pp.from_seqs(..., mode='sequential')` creates a pool that cycles through its sequences in order. Here it has 2 states: state 0 yields `'AAAAA'`, state 1 yields `'TTTTT'`.
- `pp.replacement_scan()` replaces a segment of the input sequence with an insert at each specified position. With `mode='sequential'`, it systematically enumerates all position states. Since `ins_pool` also has sequential states, the two are combined via Cartesian product: 2 inserts $\times$ 4 positions $=$ 8 total states.

  `[PLACEHOLDER: example2_rep_states.png showing the 8 replacement states]`

- `pp.deletion_scan()` removes a segment of specified length at each position, replacing it with deletion markers (`'-'`). With 4 positions, this yields 4 states.

  `[PLACEHOLDER: example2_del_states.png showing the 4 deletion states]`

- `pp.stack()` combines pools as a union: only one pool is "active" at any given state. The first 8 states come from `rep_pool`, the next 4 from `del_pool`, for a total of 12 states.
- `generate_seqs(num_complete_iterations=1)` iterates through all 12 states exactly once. Unlike `num_seqs=N` which samples N sequences (potentially with repetition for random operations), `num_complete_iterations=N` guarantees that every state is visited exactly N times.

- The resulting sequences all have the same length because `replacement_scan` replaces 5 nucleotides with a 5-nucleotide insert, and `deletion_scan` replaces 5 nucleotides with 5 deletion markers.

  `[PLACEHOLDER: example2_seqs.png showing all 12 output sequences]`

- The computation tree for this example shows how the counters are composed:

  `[PLACEHOLDER: example2_tree.png showing the computation tree with counter states]`

- The design card columns in the output dataframe track which state each operation was in when generating each sequence, enabling downstream analysis:

  `[PLACEHOLDER: example2_df.png showing the design card columns]`

The remainder of this document describes the architecture and operations of PoolParty and its underlying StateCounter library in detail.

2 StateCounter Architecture

[Write description of StateCounter's core architecture, including the Counter class, Manager context, and unidirectional state propagation mechanism.]

2.1 Core Concepts

[Describe core concepts: leaf counters, derived counters, state propagation, and conflict detection.]

2.2 The Manager Context

[Explain the Manager context manager and its role in the StateCounter system.]

3 StateCounter Operations

[Write overview of StateCounter operations and how they combine counters to create complex enumeration patterns.]

4 PoolParty Architecture

[Write description of PoolParty's core architecture, including the Pool class, lazy evaluation, and how Pools compose to define oligonucleotide libraries.]

4.1 Pool Composition

[Describe how Pool objects can be composed using string-like operations.]

4.2 Generation Modes

[Explain the different sequence generation modes available in PoolParty.]

Table 1: StateCounter Operations

Operation	Syntax	Description
Product	`A * B`	Cartesian product of two counters
Stack	`stack([A, B])`	Union/sum of counters (one active at a time)
Slice	`A[start:stop:step]`	Subset of counter states
Repeat	`A * n`	Repeat counter n times
Shuffle	`shuffle_counter(A)`	Randomly permute state order
Sample	`sample_counter(A, k)`	Random sample of k states
Synchronize	`synchronize_counters(A, B)`	Lock counters to same state
Split	`split_counter(A, sizes)`	Split counter into parts
Interleave	`interleave_counters([A, B])`	Interleave states from counters
Passthrough	`passthrough(A)`	Identity operation (for debugging)

5 PoolParty Operations

[Write overview of PoolParty operations and how they enable different library design patterns.]

Table 2: PoolParty Operations

Operation	Category	Description
`from_seq`	Creation	Create pool from a single sequence
`from_seqs`	Creation	Create pool from multiple sequences
`from_iupac_motif`	Creation	Create pool from IUPAC ambiguity codes
`from_prob_motif`	Creation	Create pool from position weight matrix
`get_kmers`	Creation	Generate all k-mers from alphabet
`deletion_scan`	Scanning	Systematic single-position deletions
`insertion_scan`	Scanning	Systematic single-position insertions
`replacement_scan`	Scanning	Systematic single-position replacements
`breakpoint_scan`	Scanning	Scan for breakpoint positions
`marker_scan`	Marker	Scan for marker positions
`marker_multiscan`	Marker	Scan multiple markers
`replace_marker`	Marker	Replace marker with sequence
`mutagenize`	Mutagenesis	Apply mutations to sequence
`mutagenize_orf`	Mutagenesis	Apply codon-aware mutations to ORF
`join`	Manipulation	Join multiple pools
`stack`	Manipulation	Stack pools (one active at a time)
`repeat`	Manipulation	Repeat pool sequence
`seq_slice`	Manipulation	Slice sequence positions
`seq_shuffle`	Manipulation	Shuffle sequence order
`reverse_complement`	Manipulation	Reverse complement sequence

(continued on next page)

(continued from previous page)

Operation	Category	Description
`state_slice`	State	Slice pool states
`state_shuffle`	State	Shuffle pool states
`state_sample`	State	Sample pool states
`sync`	State	Synchronize pool states

6 Integration Between PoolParty and StateCounter

[Write description of how PoolParty uses StateCounter internally, including how Pool operations wrap Counter operations and how state propagation enables lazy sequence generation.]

7 Code Examples

7.1 StateCounter Example

The following example demonstrates basic StateCounter usage with the product operation:

```
from statecounter import Counter, Manager

with Manager():
    # Create leaf counters
    A = Counter(num_states=2, name='A')
    B = Counter(num_states=3, name='B')

    # Combine with product (Cartesian product)
    C = A * B  # 6 states total

    # Iterate and see parent states update
    for state in C:
        print(f"C={state}, A={A.state}, B={B.state}")
```

To create a counter, use `Counter(num_states=n, name='X')`.

7.2 PoolParty Example

The following example demonstrates creating a mutagenesis library with barcodes:

```
from poolparty import Pool

# Define the DNA sequence
seq = "ATCGATCGATCGATCGATCG"

# Create a pool with all single deletions
pool = Pool.from_seq(seq).deletion_scan()

# Add a barcode region
```

```
barcode = Pool.get_kmers(length=10, alphabet='ACGT')

# Concatenate to create final library
library = pool + barcode

# Generate sequences
seqs_df = library.generate_library(
    combinatorially_complete_pools=[pool],
    num_complete_iterations=5
)
```

7.3 Advanced Example: Combining Operations

```
# Example: Create a library with multiple scanning operations
# [Add your advanced example here]
pass
```